



**VISVESVARAYA TECHNOLOGICAL UNIVERSITY**  
**JNANA SANGAMA, BELGAVI-590018, KARNATAKA**

**T. JOHN INSTITUTE OF TECHNOLOGY**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

---

## **Database Management System** **(BCS403)**

**IV Semester**

**LAB MANUAL**

**(Effective for the Academic Year 2024 -2025)**

## Department Vision

To achieve excellence in delivering quality education of global standard, coupled, with innovative practices using advanced technology and expertise, transforming the student community into potential global leaders with accountability to meet societal, national and dynamic global challenges.

## Department Mission

To develop as a top-notch department of excellence in computer science and engineering in terms of education, training, research and teaching environment that will mould student into innovative engineers competent at solving real world challenges in industry with excellent technical skills and high ethical standards.

## Program Educational Objectives (PEOs)

PEO1: Demonstrate individual personality/attributes, professional attitude, and ethics while providing solutions in professional, societal and environmental contexts.

PEO2: Develop industry focused skills, ethics and leadership qualities to become successful engineers and entrepreneurs in multidisciplinary domains.

PEO3: Demonstrate life-long learning attitude by adapting to emerging technologies.

## Program Specific Outcomes (PSOs) for BE

|       |                                                                                                                                                                                   |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PSO1: | Ability to demonstrate Ingenuity through the design, develop & analysis of computer-based systems to accomplish proficient solutions for emerging challenge in Digital ecosystem. |
| PSO2: | Ability to develop and evaluate data processing and computer communication systems to arrive at optimistic solutions in the field of information technology.                      |

## PROGRAM OUTCOMES (POs)

Engineering Graduates should possess the following:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design / development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multi-disciplinary environments.
12. **Life- long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

## DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

### General Laboratory Instructions

1. Students are advised to come to the laboratory at least 5 minutes before (to the starting time), those who come after 5 minutes will not be allowed into the lab.
2. Plan your task properly much before to the commencement, come prepared to the lab with the synopsis / program/ experiment details.
3. Student should enter into the laboratory with:
4. Laboratory observation notes with all the details (Problem statement, Aim, Algorithm, Procedure, Program, Expected Output, etc.,) filled in for the lab session.
5. Laboratory Record updated up to the last session experiments and other utensils (if any) needed in the lab.
6. Proper Dress code and Identity card.
7. Sign in the laboratory login register, write the TIME-IN, and occupy the computer system allotted to you by the faculty.
8. Execute your task in the laboratory, and record the results / output in the lab observation note book, and get certified by the concerned faculty.
9. All the students should be polite and cooperative with the laboratory staff, must maintain the discipline and decency in the laboratory.
10. Computer labs are established with sophisticated and high-end branded systems, which should be utilized properly.
11. Students / Faculty must keep their mobile phones in SWITCHED OFF mode during the lab sessions. Misuse of the equipment, misbehaviors with the staff and systems etc., will attract severe punishment.
12. Students must take the permission of the faculty in case of any urgency to go out; if anybody found loitering outside the lab / class without permission during working hours will be treated seriously and punished appropriately.
13. Students should LOG OFF/ SHUT DOWN the computer system before he/she leaves the lab after completing the task (experiment) in all aspects. He/she must ensure the system / seat is kept properly.

| DATABASE MANAGEMENT SYSTEM     |                                  | Semester    | 4   |
|--------------------------------|----------------------------------|-------------|-----|
| Course Code                    | BCS403                           | CIE Marks   | 50  |
| Teaching Hours/Week (L:T:P: S) | 3:0:2:0                          | SEE Marks   | 50  |
| Total Hours of Pedagogy        | 40 hours Theory + 8-10 Lab slots | Total Marks | 100 |
| Credits                        | 04                               | Exam Hours  |     |
| Examination nature (SEE)       | Theory                           |             |     |

**PRACTICAL COMPONENT OF IPCC (May cover all / major modules)**

| Sl.NO | Experiments                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Create a table called Employee & execute the following.<br><b>Employee(EMPNO,ENAME,JOB, MANAGER_NO, SAL, COMMISSION)</b><br>1. Create a user and grant all permissions to theuser.<br>2. Insert the any three records in the employee table contains attributes EMPNO,ENAME JOB, MANAGER_NO, SAL, COMMISSION and use rollback. Check the result.<br>3. Add primary key constraint and not null constraint to the employee table.<br>4. Insert null values to the employee table and verify the result. |
| 2     | Create a table called Employee that contain attributes EMPNO,ENAME,JOB, MGR,SAL & execute the following.<br>1. Add a column commission with domain to the Employee table.<br>2. Insert any five records into the table.<br>3. Update the column details of job<br>4. Rename the column of Employ table using alter command.<br>5. Delete the employee whose Empno is 105.                                                                                                                              |
| 3     | Queries using aggregate functions(COUNT,AVG,MIN,MAX,SUM),Group by,Orderby.<br><b>Employee(E_id, E_name, Age, Salary)</b><br>1. Create Employee table containing all Records E_id, E_name, Age, Salary.<br>2. Count number of employee names from employeetable<br>3. Find the Maximum age from employee table.<br>4. Find the Minimum age from employeetable.<br>5. Find salaries of employee in Ascending Order.<br>6. Find grouped salaries of employees.                                            |
| 4     | Create a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old & new Salary.<br><b>CUSTOMERS(ID,NAME,AGE,ADDRESS,SALARY)</b>                                                                                                                                                                                                                           |
| 5     | Create cursor for Employee table & extract the values from the table. Declare the variables ,Open the cursor & extrect the values from the cursor. Close the cursor.<br><b>Employee(E_id, E_name, Age, Salary)</b>                                                                                                                                                                                                                                                                                     |
| 6     | Write a PL/SQL block of code using parameterized Cursor, that will merge the data available in the newly created table N_RollCall with the data available in the table O_RollCall. If the data in the first table already exist in the second table then that data should be skipped.                                                                                                                                                                                                                  |
| 7     | Install an Open Source NoSQL Data base MangoDB & perform basic CRUD(Create, Read, Update & Delete) operations. Execute MangoDB basic Queries using CRUD operations.                                                                                                                                                                                                                                                                                                                                    |

**Course outcomes (Course Skill Set):**

At the end of the course, the student will be able to:

- Describe the basic elements of a relational database management system
- Design entity relationship for the given scenario.
- Apply various Structured Query Language (SQL) statements for database manipulation.
- Analyse various normalization forms for the given application.
- Develop database applications for the given real world problem.
- Understand the concepts related to NoSQL databases.

**Assessment Details (both CIE and SEE)**

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

#### **CIE for the practical component of the IPCC**

- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to **15 marks**.
- The laboratory test (**duration 02/03 hours**) after completion of all the experiments shall be conducted for 50 marks and scaled down to **10 marks**.
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

#### **SEE for IPCC**

Theory SEE will be conducted by University as per the scheduled timetable, with common question papers for the course (**duration 03 hours**)

1. The question paper will have ten questions. Each question is set for 20 marks.
2. There will be 2 questions from each module. Each of the two questions under a module (with a maximum of 3 sub-questions), **should have a mix of topics** under that module.
3. The students have to answer 5 full questions, selecting one full question from each module.
4. Marks scored by the student shall be proportionally scaled down to 50 Marks

**The theory portion of the IPCC shall be for both CIE and SEE, whereas the practical portion will have a CIE component only. Questions mentioned in the SEE paper may include questions from the practical component.**

#### **Course outcomes (Course Skill Set):**

At the end of the course, the student will be able to:

- ➔ Describe the basic elements of a relational database management system
- ➔ Design entity relationship for the given scenario.
- ➔ Apply various Structured Query Language (SQL) statements for database manipulation.
- ➔ Analyse various normalization forms for the given application.
- ➔ Develop database applications for the given real-world problem.
- ➔ Understand the concepts related to NoSQL databases

## BCS403: DBMS LABORATORY

**Database:** A Database is a collection of interrelated data and a Database Management System is a software system that enables users to define, create and maintain the database and which provides controlled access to the database

**SQL:** It is structured query language, basically used to pass the query to retrieve and manipulate the information from database. Depending upon the nature of query, SQL is divided into different components:

- **DDL**(Data Definition Language )
- **DML**(Data Manipulation Language )
- **DCL**(Data Control Language )

**DDL:** The Data Definition Language (DDL) is used to create the database (i.e. tables, keys, relationships etc), maintain the structure of the database and destroy databases and database objects.

**Eg.** Create, Drop, Alter, Describe, Truncate

1. **CREATE statements:** It is used to create the table.

**Syntax:**

CREATE TABLE table\_name(columnName1 datatype(size), columnName2 datatype(size), ..... );

2. **DROP statements:** To destroy an existing database, table, index, or view. If a table is dropped all records held within it are lost and cannot be recovered.

**Syntax:**

DROP TABLE table\_name;

3. **ALTER statements:** To modify an existing database object.

• **Adding new columns:**

**Syntax:**

Alter table table\_name Add(New\_columnName1 datatype(size), New\_columnName2 datatype(size), ..... )*DBMS Lab Manual*

• **Dropping a columns from a table :**

**Syntax:**

Alter table table\_name DROP column columnName;

• **Modifying Existing columns:**

**Syntax:**

Alter table table\_name Modify (columnName1 Newdatatype(Newsize));

4. **Describe statements:** To describe the structure (column and data types) of an existing database, table, index, or view.

**Syntax:**

DESC table\_name;

5. **Truncate statements:** To destroy the data in an existing database, table, index, or view. If a table is truncated all records held within it are lost and cannot be recovered but the table structure is maintained.

**Syntax :**

TRUNCATE TABLE table\_name;

### **Data Manipulation Language (DML):**

- A Data Manipulation Language enables programmers and users of the database to retrieve insert, delete and update data in a database. e.g. INSERT, UPDATE, DELETE, SELECT.

**INSERT:** INSERT statement adds one or more records to any single table in a relational database.

**Syntax:**

INSERT INTO tablename VALUES (expr1, expr2.....);

**UPDATE:** UPDATE statement that changes the data of one or more records in a table. Either all the rows can be updated, or a subset may be chosen using a condition.

**Syntax:**

UPDATE table\_name SET column\_name = value [, column\_name = value....] [WHERE condition]

**DELETE:** DELETE statement removes one or more records from a table. A subset may be defined for deletion using a condition, otherwise all records are removed.

**Syntax:**

DELETE FROM tablename WHERE condition;

**SELECT:** SELECT statement returns a result set of records from one or more tables.

The select statement has optional clauses:

- WHERE specifies which rows to retrieve
- GROUP BY groups rows sharing a property so that an aggregate function can be applied to each group having group.
- HAVING selects among the groups defined by the GROUP BY clause.
- ORDER BY specifies an order in which to return the rows.

**Syntax:**

SELECT<attribute list> FROM<table list>

WHERE<condition> Where

- Attribute list is a list of attribute name whose values to be retrieved by the query.
- Table list is a list of table name required to process query.
- Condition is a Boolean expression that identifies the tuples to be retrieved by query.

**Data Constraints** are the business Rules which are enforced on the data being stored in a table are called Constraints.

Types of Data Constraints

1. I/O Constraint This type of constraint determines the speed at which data can be inserted or extracted from an Oracle table. I/O Constraints is divided into two different types
  - The Primary Key Constraint
  - The Foreign Key Constraint
2. Business rule Constraint This type of constraint is applied to data prior the data being
3. Inserted into table columns.
  - Column level
  - Table level

### **The PRIMARY KEY defined at column level**

**Syntax:**

```
CREATETABLE tablename (Columnname1DATATYPE
    CONSTRAINT <constraintname1>
    PRIMARY KEY, Columnname2
    DATATYPE, columnname3
    DATATYPE, ....);
```

**The PRIMARY KEY defined at table level****Syntax:**

```
CREATE TABLE tablename (Columnname1 DATATYPE, columnname2 DATATYPE,  
columnname3 DATATYPE, PRIMARY KEY (columnname1, columnname2));
```

**The FOREIGN KEY defined at column level****Syntax**

```
CREATE  
TABLE tablename (Columnname1 tablename[(columnname)] [ON DELETE CASCADE],  
columnname3 DATATYPE ,.... );  
DATATYPE columnname2 REFERENCES DATATYPE ,
```

The table in which FOREIGN KEY is defined is called FOREIGN TABLE or DETAIL TABLE.  
The table in which PRIMARY KEY is defined and referenced by FOREIGN KEY is called  
PRIMARY TABLE or MASTER TABLE.

**ON DELETE CASCADE** is set then DELETE operation in master table will trigger the  
DELETE operation for corresponding records in the detail table.

**The FOREIGN KEY defined at table level****Syntax:**

```
CREATE TABLE tablename (Columnname1 DATATYPE, columnname2  
DATATYPE, columnname3 DATATYPE, PRIMARY KEY (columnname1, columnname2),  
FOREIGN KEY (columnname2) REFERENCES tablename2;
```

**A CONSTRAINT** can be given User

Defined Name, the syntax is:

```
CONSTRAINT <constraint name><constraint definition>
```

**The CHECK Constraint defined at column level****Syntax:**

```
CREATE TABLE tablename (Columnname1 DATATYPE CHECK (logical expression),  
columnname2 DATATYPE, columnname3 DATATYPE, .. );
```

**The CHECK Constraint defined at table level****Syntax:**

```
CREATE TABLE tablename (Columnname1 DATATYPE, columnname2 DATATYPE,  
columnname3 DATATYPE, CHECK (logical expression1), CHECK (logical expression2));
```

**The UNIQUE Constraint defined at the column level****Syntax:**

```
CREATE TABLE tablename (Columnname1 DATATYPE UNIQUE, columnname2  
DATATYPE  
UNIQUE, columnname3 DATATYPE ....);
```

**The UNIQUE Constraint defined at the the table level****Syntax:**

```
CREATE TABLE tablename (Columnname1 DATATYPE, columnname2 DATATYPE,  
columnname3 DATATYPE, UNIQUE(columnname1));
```

**NOT NULL constraint defined at column level:****Syntax:**

```
CREATE TABLE tablename (Columnname1 DATATYPE NOT NULL, columnname2
```

DATATYPE NOT NULL, columnname3 DATATYPE,...);

**Note:**

The NOT NULL constraint can only be applied at column level.

**ER- Diagram:** It is an Entity –Relationship diagram which is used to represent the relationship between different entities. An entity is an object in the real world which is distinguishable from other objects. The overall logical structure of a database can be expressed graphically by an ER diagram, which is built up from following components.

- Rectangles: represent entity sets.
- Ellipses: represent attributes.
- Diamonds: represent relationships among entity sets.
- Lines: link attribute to entity sets and entity sets to relationships.

**Mapping Cardinalities:** It expresses the number of entities to which another entity can be associated via a relationship set. For a binary relationship set R between entity sets A and B. The Mapping Cardinalities must be one of the following.

- One to one
- One to many
- Many to one
- Many to many

## LAB EXPERIMENTS

### PROGRAM 1

**AIM:** Create a table called Employee & execute the following.

Employee (EMPNO, ENAME, JOB, MANAGER\_NO, SAL, COMMISSION)

1. Create a user and grant all permissions to the user.
2. Insert the any three records in the employee table contains attributes EMPNO, ENAME JOB, MANAGER\_NO, SAL, COMMISSION and use rollback. Check the result.
3. Add primary key constraint and not null constraint to the employee table.
4. Insert null values to the employee table and verify the result.

### PROGRAM CODE:

#### 1. Create a user and grant all permissions to the user

```
CREATE USER myuser IDENTIFIED BY mypassword;  
GRANT ALL PRIVILEGES ON Employee TO myuser;
```

#### 2. Create the Employee table

```
CREATE TABLE Employee (  
EMPNO INT,  
ENAME VARCHAR(50),  
JOB VARCHAR(50),  
MANAGER_NO INT,  
SAL DECIMAL(10, 2),  
COMMISSION DECIMAL(10, 2)  
);
```

#### 2. Insert three records into the employee table and rollback

```
INSERT INTO Employee (EMPNO, ENAME, JOB, MANAGER_NO, SAL, COMMISSION)  
VALUES  
(1, 'John Doe', 'Manager', NULL, 5000.00, 1000.00),  
(2, 'Jane Smith', 'Developer', 1, 4000.00, 500.00),  
(3, 'Bob Johnson', 'Analyst', 1, 3500.00, NULL);  
ROLLBACK;  
Check if the records were inserted  
SELECT * FROM Employee;
```

#### 3. Add primary key constraint and not null constraint to the employee table

```
ALTER TABLE Employee  
ADD CONSTRAINT PK_Employee PRIMARY KEY (EMPNO);  
ALTER TABLE Employee  
MODIFY (EMPNO INT NOT NULL,  
ENAME VARCHAR(50) NOT NULL,  
JOB VARCHAR(50) NOT NULL,  
SAL DECIMAL(10, 2) NOT NULL);
```

#### 4. Insert null values into the employee table and verify the result

```
INSERT INTO Employee (EMPNO, ENAME, JOB, MANAGER_NO, SAL, COMMISSION)
VALUES (4, NULL, 'Intern', NULL, NULL, NULL);
SELECT * FROM Employee;
```

#### OUTPUT:

| EMPNO | ENAME       | JOB       | MANAGER_NO | SAL   | COMMISSION |
|-------|-------------|-----------|------------|-------|------------|
| 1     | John Doe    | Manager   | NULL       | 50000 | 10000      |
| 2     | Jane Smith  | Developer | 1          | 25000 | 5000       |
| 3     | Bob Johnson | Analyst   | 1          | 30000 | NULL       |
| 4     | NULL        | Intern    | NULL       | NULL  | NULL       |

#### PROGRAM 2

**AIM:** Create a table called Employee that contain attributes EMPNO, ENAME, JOB, MGR, SAL and execute the following.

1. Add a column commission with domain to the Employee table.
2. Insert any five records into the table.
3. Update the column details of job
4. Rename the column of Employ table using alter command.
5. Delete the employee whose Empno is 105.

#### 1. Create the Employee table with attributes EMPNO, ENAME, JOB, MGR, SAL

```
CREATE TABLE Employee (
EMPNO INT,
ENAME VARCHAR(50),
JOB VARCHAR(50),
MGR INT,
SAL DECIMAL(10, 2)
);
```

#### 2. Add a column 'commission' to the Employee table

```
ALTER TABLE Employee
ADD commission DECIMAL (10, 2);
```

#### 3. Insert five records into the table

```
INSERT INTO Employee (EMPNO, ENAME, JOB, MGR, SAL, commission)
VALUES
(101, 'John Doe', 'Manager', NULL, 5000.00, 1000.00),
(102, 'Jane Smith', 'Developer', 101, 4000.00, 500.00),
(103, 'Bob Johnson', 'Analyst', 101, 3500.00, NULL),
(104, 'Alice Jones', 'Designer', 101, 3800.00, 800.00),
(105, 'Michael Brown', 'Engineer', 101, 4200.00, 700.00);
```

#### 4. Update the column details of 'JOB'

```
ALTER TABLE Employee
MODIFY (JOB VARCHAR(50) NOT NULL);
```

#### 5. Rename the column 'MGR' to 'MANAGER' in the Employee table

```
ALTER TABLE Employee
RENAME COLUMN MGR TO MANAGER;
```

#### 6. Delete the employee whose Empno is 105

```
DELETE FROM Employee WHERE EMPNO = 105;
Select*from Employee;
```

#### OUTPUT:

| EMPNO | ENAME       | JOB       | MANAGER_NO | SAL   | COMMISSION |
|-------|-------------|-----------|------------|-------|------------|
| 101   | John Doe    | Manager   | NULL       | 50000 | 10000      |
| 102   | Jane Smith  | Developer | 101        | 25000 | 5000       |
| 103   | Bob Johnson | Analyst   | 101        | 30000 | NULL       |
| 104   | Alice Jones | Designer  | 101        | 34000 | 2000       |

### PROGRAM 3

**AIM:** Queries using aggregate functions (COUNT, AVG, MIN, MAX, SUM), Group by, Orderby.  
Employee (E\_id, E\_name, Age, Salary)

1. Create Employee table containing all Records E\_id, E\_name, Age, Salary.
2. Count number of employee names from employee table
3. Find the Maximum age from employee table.
4. Find the Minimum age from employee table.
5. Find salaries of employee in Ascending Order.
6. Find grouped salaries of employees.

#### 1. Create the Employee table

```
CREATE TABLE Employee (E_id INT,E_name VARCHAR(50),Age INT,Salary DECIMAL(10, 2));
```

**OUTPUT:** Table created

#### 2. Insert records into the Employee table

```
INSERT INTO Employee (E_id, E_name, Age, Salary) VALUES
(1, 'John Doe', 30, 50000.00), (2, 'Jane Smith', 25, 45000.00), (3, 'Bob Johnson', 35, 60000.00), (4,
'Alice Jones', 28, 52000.00), (5, 'Michael Brown', 32, 55000.00);
```

```
Select * from Employee;
```

#### 3.Count number of employee names from employee table

```
SELECT COUNT(E_name) AS num_employees FROM Employee;
```

**OUTPUT:** 5

#### 4. Find the Maximum age from employee table

SELECT MAX(Age) AS max\_age FROM Employee;

**OUTPUT:** 35

#### 5. Find the Minimum age from employee table

SELECT MIN(Age) AS min\_age FROM Employee;

**OUTPUT:** 25

#### 6. Find salaries of employee in Ascending Order

SELECT E\_name, Salary FROM Employee ORDER BY Salary ASC;

**OUTPUT:**

|               |          |
|---------------|----------|
| Jane Smith    | 45000.00 |
| John Doe      | 50000.00 |
| Alie Jones    | 52000.00 |
| Michael Brown | 55000.00 |
| Bob Johnson   | 60000.00 |

#### 7. Find grouped salaries of employees

SELECT Salary, COUNT(\*) AS num\_employees FROM Employee GROUP BY Salary;

**OUTPUT:**

|          |   |
|----------|---|
| 50000.00 | 1 |
| 45000.00 | 1 |
| 60000.00 | 1 |
| 52000.00 | 1 |
| 55000.00 | 1 |

### PROGRAM 4

**AIM:** Create a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old & new Salary.CUSTOMERS(ID,NAME,AGE,ADDRESS,SALARY)

```
Create the customers table
CREATE TABLE customers (
ID INT PRIMARY KEY,
NAME VARCHAR(50),
AGE INT,
ADDRESS VARCHAR(100),
SALARY DECIMAL(10, 2)
);
-- Create a sequence for trigger
CREATE SEQUENCE salary_diff_seq;
-- Create the trigger
CREATE OR REPLACE TRIGGER salary_diff_trigger
```

## AFTER INSERT OR UPDATE OR DELETE ON customers

FOR EACH ROW

DECLARE

old\_salary DECIMAL(10, 2);

new\_salary DECIMAL(10, 2);

salary\_diff DECIMAL(10, 2);

BEGIN

-- Get the old and new salary values

IF INSERTING OR UPDATING THEN

old\_salary := NVL(:OLD.SALARY, 0);

new\_salary := NVL(:NEW.SALARY, 0);

END IF;

IF DELETING THEN

old\_salary := NVL(:OLD.SALARY, 0);

new\_salary := 0;

END IF;

-- Calculate the salary difference

salary\_diff := new\_salary - old\_salary;

-- Display the salary difference

DBMS\_OUTPUT.PUT\_LINE('Salary difference for ID ' || :OLD.ID || ' : ' || salary\_diff);

-- Increment sequence

SELECT salary\_diff\_seq.NEXTVAL INTO NULL FROM DUAL;

END;

/

## PROGRAM 5

**AIM:** Create cursor for Employee table & extract the values from the table. Declare the variables ,Open the cursor & extract the values from the cursor. Close the cursor.

Employee(E\_id, E\_name, Age, Salary).

DECLARE

-- Declare variables to hold values from the cursor

v\_E\_id Employee.E\_id%TYPE;

v\_E\_name Employee.E\_name%TYPE;

v\_Age Employee.Age%TYPE;

v\_Salary Employee.Salary%TYPE;

-- Declare cursor for the Employee table

CURSOR emp\_cursor IS

SELECT E\_id, E\_name, Age, Salary

FROM Employee;

BEGIN

-- Open the cursor

OPEN emp\_cursor;

-- Fetch and process each row from the cursor

LOOP

```

FETCH emp_cursor INTO v_E_id, v_E_name, v_Age, v_Salary;
EXIT WHEN emp_cursor%NOTFOUND;
-- Process the values, for example, you can print them
DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_E_id || ', Name: ' || v_E_name || ', Age: ' ||
v_Age || ', Salary: ' || v_Salary);
END LOOP;
-- Close the cursor
CLOSE emp_cursor;
END;
/

```

## PROGRAM 6

**AIM:** Write a PL/SQL block of code using parameterized Cursor, that will merge the data available in the newly created table N\_RollCall with the data available in the table O\_RollCall. If the data in the first table already exist in the second table, then that data should be skipped.

```

create database sql6;
use sql6;
create table o_rollcall(roll_no int,name varchar(20),address varchar(20));
create table n_rollcall(roll_no int,name varchar(20),address varchar(20));
insert into o_rollcall values('1','Hitesh','Nandura');
insert into o_rollcall values('2','Piyush','MP');
insert into o_rollcall values('3','Ashley','Nsk');
insert into o_rollcall values('4','Kalpesh','Dhule');
insert into o_rollcall values('5','Abhi','Satara');
delimiter //
create procedure p3(in r1 int)
begin
declare r2 int;
declare exit_loop boolean;
declare c1 cursor for select roll_no from o_rollcall
where roll_no>r1;
declare continue handler for not found set
exit_loop=true;
open c1;
e_loop:loop
fetch c1 into r2;
if not exists(select * from n_rollcall where
roll_no=r2)
then
insert into n_rollcall select * from o_rollcall where
roll_no=r2;
end if;
if exit_loop
then
close c1;
leave e_loop;
end if;
end loop e_loop;
end
//

```

```
call p3(3);
select * from n_rollcall;
call p3(0);
select * from n_rollcall;
insert into o_rollcall values('6','Patil','Kolhapur');
call p3(4);
select * from n_rollcall;
```

#### OUTPUT:

| Roll no | Name    | Address  |
|---------|---------|----------|
| 4       | Kalpesh | Dhule    |
| 5       | Abhi    | Satara   |
| 1       | Hitesh  | Nandura  |
| 1       | Hitesh  | Nandura  |
| 2       | Piyush  | MP       |
| 3       | Ashley  | Nsk      |
| 6       | Patil   | Kolhapur |

7. Install an Open Source NoSQL Data base MangoDB & perform basic CRUD(Create, Read, Update & Delete) operations. Execute MangoDB basic Queries using CRUD operations.

#### Installation:

1. Download the MongoDB Installer:

- Visit the official MongoDB download center:
- URL: <https://www.mongodb.com/try/download/community>
- Select the appropriate version (latest stable is recommended for most cases).
- Under "Platform," choose "Windows."
- For "Package," select "MSI."
- Click "Download."

2. Run the Installer:

- Locate the downloaded .msi file (usually in your Downloads folder).
- Double-click the file to launch the MongoDB Setup Wizard.

3. Follow the Installation Wizard:

- Click "Next" on the initial screen.
- Accept the license agreement and click "Next."
- Choose the installation type:
  - o Complete: Installs all MongoDB components (recommended for most users).
  - o Custom: Allows you to select specific features (advanced users).
- Click "Next."

4. Configure Service Options (Optional):

- In the "Service Configuration" step (available in newer versions), you can choose how MongoDB runs as a service:
  - o Run service as Network Service user: This is the default and recommended option.
  - o Run service as a custom user account: If you have specific security requirements, you can create a dedicated service account.
- Click "Next."

5. Review and Install:

- Review the installation summary.
- Click "Install" to begin the installation process.

6. Verify Installation

#### Basic CRUD(Create, Read, Update & Delete) operations

Now that you've installed MongoDB Compass on your local machine, you can connect it to the MongoDB instance running on your remote server.

Create new database "mflix"

Create a collection "movies"

### Create Operation

Insert data into the collection.

Click on "Add Data". There will be two options:

1. Import JSON/CSV file
2. Insert Document

Click on "Insert Document" and enter the record details:

```
{
  "_id": {},
  "title": "Jurassic World: Fallen Kingdom",
  "genres": [],
  "runtime": 130,
  "rated": "PG-13",
  "year": 2018,
  "directors": [],
  "cast": [],
  "type": "movie"
}
```

|   | _id ObjectId                  | title String               | genres Array   | runtime Int32 | rated String |                                                                                                                                                                                                                                                                   |
|---|-------------------------------|----------------------------|----------------|---------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | ObjectId('66164b31992bb9...') | "Jurassic World: Fallen .. | [ ] 2 elements | 130           | "PG-13"      |          |
| 2 | ObjectId('66164c54992bb9...') | No field                   | No field       | No field      | No field     |       |
| 3 | ObjectId('66164d6e992bb9...') | "Avatar"                   | [ ] 3 elements | 162           | "PG-13"      |    |
| 4 | ObjectId('66164e12992bb9...') | "Friends"                  | [ ] 1 elements | 10            | "TV-PG"      |    |
| 5 | ObjectId('66164f6a992bb9...') | No field                   | No field       | No field      | No field     |    |
| 6 | ObjectId('66165024992bb9...') | No field                   | No field       | No field      | No field     |    |

### Read Operation

Click on a specific collection to view its contents.

To filter the contents, enter the query in the filter box.





ADD DATA
EXPORT DATA
UPDATE
DELETE

```

_id: ObjectId('66164f6a992bb9d7db5e675e')
Title: "Frozen"
Genres: Array (4)
Runtime: 102
Rated: "PG"
Year: "2013"
Director: Array (2)
Cast: Array (2)
```

### Update Operation

Select the desired collection and click on Update button to modify the values of a record. Set new values for the desired fields.

## Update 1 document

mflix.movies

Filter ⓘ

{ Title: 'Frozen' }

Update

[Learn more about Update syntax](#)

```
1 {
2   $set: { Runtime: 103
3
4   },
5 }
```

★ Save

Cancel

Update 1 document

## Delete Operation

Click on the Delete button to delete records of a specific collection.  
 To delete a specific record, filter the record and then delete.

## Delete 6 documents

mflix.movies

Filter ⓘ

{}

Export

Preview (sample of 5 documents)

```
{
  _id: ObjectId('66164b31992bb9d7db5e6755'),
  title: "Jurassic World: Fallen Kingdom",
  genres: Array (2)
  runtime: 130
  rated: "PG-13"
  year: 2018
  directors: Array (1)
  cast: Array (3)
  type: "movie"
}
```

```
{
  _id: ObjectId('66164c54992bb9d7db5e675a'),
  Title: "Home Alone"
  Genres: Array (2)
}
```

Cancel

Delete 6 documents

**Viva Questions****1. What is SQL?**

Structured Query Language

**2. What is database?**

A database is a logically coherent collection of data with some inherent meaning, representing some aspect of real world and which is designed, built and populated with data for a specific purpose.

**3. What is DBMS?**

It is a collection of programs that enables user to create and maintain a database. In other words it is general-purpose software that provides the users with the processes of defining, constructing and manipulating the database for various applications.

**4. What is a Database system?**

The database and DBMS software together is called as Database system.

**5. Advantages of DBMS?**

- Redundancy is controlled.
- Unauthorized access is restricted.

**6. What is the difference between SQL and PL-SQL?**

SQL is a declarative language. PL/SQL is a procedural language

**7. Describe the three levels of data abstraction?**

There are three levels of abstraction:

- Physical level: The lowest level of abstraction describes how data are stored.
- Logical level: The next higher level of abstraction, describes what data are stored in database and what relationship among those data.
- View level: The highest level of abstraction describes only part of entire database.

**8. Define the "integrity rules"**

There are two Integrity rules.

- Entity Integrity: States that "Primary key cannot have NULL value"
- Referential Integrity: States that "Foreign Key can be either a NULL value or should be Primary Key value of other relation.

**9. What is extension and intension?**

Extension - It is the number of tuples present in a table at any instance. This is time dependent.

Intension -It is a constant value that gives the name, structure of table and the constraints laid on it.

**10. What is Data Independence?**

Data independence means that "the application is independent of the storage structure and access strategy of data". In other words, The ability to modify the schema definition in one level should not affect the schema definition in the next higher level.

Two types of Data Independence:

- Physical Data Independence: Modification in physical level should not affect the logical level.
- Logical Data Independence: Modification in logical level should affect the view level.

NOTE: Logical Data Independence is more difficult to achieve

**11. What is a view? How it is related to data independence?**

A view may be thought of as a virtual table, that is, a table that does not really exist in its own right but is instead derived from one or more underlying base table. In other words, there is no stored file that direct represents the view instead a definition of view is stored in data dictionary.

Growth and restructuring of base tables is not reflected in views. Thus the view can insulate users from the effects of restructuring and growth in the database. Hence accounts for logical data independence.

**12. What is Data Model?**

A collection of conceptual tools for describing data, data relationships data semantics and constraints.

**13. What is E-R model?**

This data model is based on real world that consists of basic objects called entities and of relationship among these objects. Entities are described in a database by a set of attributes.

**14. What is Object Oriented model?**

This model is based on collection of objects. An object contains values stored in instance variables within the object. An object also contains bodies of code that operate on the object.

These bodies of code are called methods. Objects that contain same types of values and the same methods are grouped together into classes.

**15. What is an Entity?**

It is an 'object' in the real world with an independent existence.

**16. What is an Entity type?**

It is a collection (set) of entities that have same attributes.

**17. What is an Entity set?**

It is a collection of all entities of particular entity type in the database.

**18. What is an Extension of entity type?**

The collections of entities of a particular entity type are grouped together into an entity set.

**19. What is an attribute?**

It is a particular property, which describes the entity.

**20. What is a Relation Schema and a Relation?**

A relation Schema denoted by  $R(A_1, A_2, \dots, A_n)$  is made up of the relation name  $R$  and the list of attributes  $A_i$  that it contains. A relation is defined as a set of tuples. Let  $r$  be the relation which contains set tuples  $(t_1, t_2, t_3, \dots, t_n)$ . Each tuple is an ordered list of  $n$  values  $t = (v_1, v_2, \dots, v_n)$ .

**21. What is degree of a Relation?**

It is the number of attribute of its relation schema.

**22. What is Relationship?**

It is an association among two or more entities.

**23. What is Relationship set?**

The collection (or set) of similar relationships.

**24. What is Relationship type?**

Relationship type defines a set of associations or a relationship set among a given set of entity types.

**25. What is degree of Relationship type?**

It is the number of entity type participating.

**26. What is DDL (Data Definition Language)?**

A data base schema is specified by a set of definitions expressed by a special language called DDL.

**27. What is VDL (View Definition Language)?**

It specifies user views and their mappings to the conceptual schema.

**28. What is SDL (Storage Definition Language)?**

This language is to specify the internal schema. This language may specify the mapping between two schemas.

**29. What is Data Storage - Definition Language?**

The storage structures and access methods used by database system are specified by a set of definition in a special type of DDL called data storage- definition language.

**30. What is DML (Data Manipulation Language)?**

This language that enable user to access or manipulate data as organized by appropriate data model.

➤ Procedural DML or Low level: DML requires a user to specify what data are needed and how to get those data.

➤ Non-Procedural DML or High level: DML requires a user to specify what data are needed without specifying how to get those data.

**31. What is DML Compiler?**

It translates DML statements in a query language into low-level instruction that the query evaluation engine can understand.

**32. What is Relational Algebra?**

It is a procedural query language. It consists of a set of operations that take one or two relations as input and produce a new relation.

**33. What is Relational Calculus?**

It is an applied predicate calculus specifically tailored for relational databases proposed by E.F. Codd. E.g. of languages based on it are DSL, ALPHA, QUEL.

### 34. What is normalization?

It is a process of analyzing the given relation schemas based on their Functional Dependencies (FDs) and primary key to achieve the properties

- Minimizing redundancy
- Minimizing insertion, deletion and update anomalies.

### 35. What is Functional Dependency?

A Functional dependency is denoted by  $X \rightarrow Y$  between two sets of attributes  $X$  and  $Y$  that are subsets of  $R$  specifies a constraint on the possible tuple that can form a relation state  $r$  of  $R$ . The constraint is for any two tuples  $t_1$  and  $t_2$  in  $r$  if  $t_1[X] = t_2[X]$  then they have  $t_1[Y] = t_2[Y]$ . This means the value of  $X$  component of a tuple uniquely determines the value of component  $Y$ .

### 36. When is a functional dependency $F$ said to be minimal?

- Every dependency in  $F$  has a single attribute for its right hand side.
- We cannot replace any dependency  $X \rightarrow Y$  in  $F$  with a dependency  $X \rightarrow A$  where  $A$  is a proper subset of  $Y$  and still have a set of dependency that is equivalent to  $F$ .

A in  $F$  with a dependency  $X \rightarrow A$  where  $A$  is a proper subset of  $Y$  and still have a set of dependency that is equivalent to  $F$ .

- We cannot remove any dependency from  $F$  and still have set of dependency that is equivalent to  $F$ .

### 37. What is Multivalued dependency?

Multivalued dependency denoted by  $X \twoheadrightarrow Y$  specified on relation schema  $R$ , where  $X$  and  $Y$  are both subsets of  $R$ , specifies the following constraint on any relation  $r$  of  $R$ : if two tuples  $t_1$  and  $t_2$  exist in  $r$  such that  $t_1[X] = t_2[X]$  then  $t_3$  and  $t_4$  should also exist in  $r$  with the following properties

- $t_3[X] = t_4[X] = t_1[X] = t_2[X]$
- $t_3[Y] = t_1[Y]$  and  $t_4[Y] = t_2[Y]$
- $t_3[Z] = t_2[Z]$  and  $t_4[Z] = t_1[Z]$

where  $Z = (R - (X \cup Y))$

### 38. What is Lossless join property?

It guarantees that the spurious tuple generation does not occur with respect to relation schemas after decomposition.

### 39. What is 1 NF (Normal Form)?

The domain of attribute must include only atomic (simple, indivisible) values.

### 40. What is Fully Functional dependency?

It is based on concept of full functional dependency. A functional dependency  $X \rightarrow Y$  is fully functional dependency if removal of any attribute  $A$  from  $X$  means that the dependency does not hold any more.

### 41. What is 2NF?

A relation schema  $R$  is in 2NF if it is in 1NF and every non-prime attribute  $A$  in  $R$  is fully functionally dependent on primary key.

### 42. What is 3NF?

A relation schema  $R$  is in 3NF if it is in 2NF and for every FD  $X \rightarrow A$  either of the following is true

- $X$  is a Super-key of  $R$ .
- $A$  is a prime attribute of  $R$ .

In other words, if every non prime attribute is non-transitively dependent on primary key.

### 43. What is BCNF (Boyce-Codd Normal Form)?

A relation schema  $R$  is in BCNF if it is in 3NF and satisfies additional constraints that for every FD  $X \rightarrow A$ ,  $X$  must be a candidate key.

### 44. What is 4NF?

A relation schema  $R$  is said to be in 4NF if for every Multivalued dependency  $X \twoheadrightarrow Y$  that holds over  $R$ , one of following is true

- $X$  is subset or equal to (or)  $XY = R$ .
- $X$  is a super key.

**45. What is 5NF?**

A Relation schema R is said to be 5NF if for every join dependency  $\{R_1, R_2, \dots, R_n\}$  that holds R, one the following is true

- $R_i = R$  for some i.
- The join dependency is implied by the set of FD, over R in which the left side is key of R.

**46. What is Domain-Key Normal Form?**

A relation is said to be in DKNF if all constraints and dependencies that should hold on the constraint can be enforced by simply enforcing the domain constraint and key constraint on the relation.